

TDD Completo

Vehiculos, Cartas, Pickups y Power-Ups

Blueprint-First para Unreal Engine 5.7

Proyecto	Dungeons Dash
Motor	Unreal Engine 5.7
Enfoque	Blueprint-first con soporte futuro para C++
Sistemas cubiertos	Vehiculo, fisicas arcade, cartas, pickups, power-ups, terreno y red
Version del documento	1.0

Indice

- 1. Objetivo general
- 2. Principio tecnico base
- 3. Arquitectura general
- 4. Estructura de carpetas
- 5. Ajustes al sistema de cartas
- 6. Reglas corregidas de cartas
- 7. Vehiculo base
- 8. Componentes del vehiculo
- 9. Componentes Blueprint
- 10. Enhanced Input
- 11. Movimiento base
- 12. Salto
- 13. Drift y mini-turbo
- 14. Terreno especial
- 15. Pickups de cartas
- 16. Inventario de cartas
- 17. GameManager / GameMode
- 18. Acciones de carta
- 19. Modificadores
- 20. Power-ups basicos
- 21-26. Detalle de cartas
- 27. Interfaces Blueprint
- 28-30. Flujos completos
- 31. Multiplayer / LAN
- 32. HUD
- 33. Estados y respawn
- 34. Orden de implementacion
- 35. Responsabilidades finales
- 36. Conclusion tecnica

1. Objetivo general

Este TDD define como implementar y conectar los sistemas principales de gameplay de Dungeons Dash usando una arquitectura Blueprint-first. El documento une el sistema de vehiculos arcade, las mecanicas principales de conduccion, los pickups flotantes de cartas, las cartas basicas, los power-ups, el inventario, los modificadores del vehiculo y la comunicacion de red.

El objetivo es que cada integrante del equipo pueda trabajar en paralelo sin que un sistema dependa demasiado de otro. El vehiculo expone funciones, las cartas ejecutan acciones, el GameManager valida y decide en servidor, y los efectos visuales se replican a los clientes mediante variables RepNotify.

- Vehiculos arcade con aceleracion, frenado, giro, salto, drift y mini-turbo.
- Terreno especial: lodo, agua y Boost Pads.
- Pickups flotantes que entregan cartas al jugador.
- Cartas basicas conectadas a Data Assets y acciones UObject Blueprint.
- Modificadores aplicados al vehiculo mediante componentes Blueprint.
- Preparacion para red LAN: cliente solicita, servidor valida y replica feedback.

2. Principio tecnico base

La implementacion debe priorizar Blueprints. C++ queda como opcion futura para optimizacion, networking avanzado o sistemas que crezcan demasiado. La prioridad actual es que el sistema sea rapido de prototipar, claro de depurar y facil de conectar con el trabajo de los demas integrantes.

Sistema	Implementacion principal
Vehiculo base	Blueprint sobre ChaosWheeledVehiclePawn
Drift y mini-turbo	Blueprint Component
Pickups de cartas	Blueprint Actor
Cartas	Data Asset + UObject Blueprint
Power-ups	UObject Action + Actors cuando haya proyectiles/trampas
Modificadores del vehiculo	Blueprint Actor Component
Terreno especial	Blueprint Actor con colisiones
HUD de cartas	Widget Blueprint

Sistema	Motivo para moverlo a C++ en el futuro
Inventario de cartas	Replicacion avanzada u optimizacion
Modifier Component	Manejo robusto de estados en multiplayer
GameManager	Reglas de carrera mas complejas
Proyectiles	Muchos actores simultaneos
Checkpoints	Precision de carrera y ordenamiento

3. Arquitectura general del sistema

La arquitectura se organiza en piezas pequenas tipo Lego. Cada pieza tiene una responsabilidad especifica y se comunica con las demas mediante interfaces, referencias de componentes o llamadas controladas por el GameManager.

Dungeons Dash - TDD Completo Blueprint-First

Jugador / Input

- > BP_DashVehicle_Base
- > BPC_CardInventory
- > Server_RequestUseCard
- > GM_RacingGame / GameManager
- > BP_Action_CardName
- > BPC_ModifierComponent / Vehiculo / Proyectoil / Trampa
- > RepNotify / VFX / HUD

- El inventario solo guarda cartas, selecciona y solicita su uso.
- El Data Asset solo guarda datos: nombre, icono, duracion, intensidad y clase de accion.
- La accion de carta contiene la regla ejecutable de la carta.
- El GameManager valida y resuelve objetivos en servidor.
- El vehiculo recibe efectos, pero no necesita saber que carta los causo.
- Los clientes muestran efectos visuales y sonoros replicados.

4. Estructura de carpetas recomendada

/Content/DungeonsDash/_Core/Input/

IMC_VehicleControls
IA_Accelerate, IA_Brake, IA_Steer, IA_Jump, IA_Drift
IA_UseCard, IA_SelectCardLeft, IA_SelectCardRight

/Content/DungeonsDash/_Core/BlueprintInterfaces/

BPI_CardInventoryReceiver
BPI_VehicleEffects
BPI_HitReceiver
BPI_RacePositionProvider

/Content/DungeonsDash/_Core/Data/Cards/

DA_Card_Base
DA_Card_SpeedPotion
DA_Card_Fireball
DA_Card_HomingArrow
DA_Card_ArcaneShield
DA_Card_ExplosiveBarrel
DA_Card_ComebackBlessing

/Content/DungeonsDash/_Core/Data/Enums/

E_CardType
E_CardRarity
E_DashModifierType
E_DashSurfaceType
E_DashVehicleState

/Content/DungeonsDash/Actors/Vehicles/

BP_DashVehicle_Base
BP_DashVehicle_Knight
BP_DashVehicle_Mage
BP_DashVehicle_Barbarian

Dungeons Dash - TDD Completo Blueprint-First

/Content/DungeonsDash/Actors/Vehicles/Components/

BPC_CardInventory
BPC_ModifierComponent
BPC_VehicleDrift
BPC_VehicleSurfaceHandler
BPC_VehicleRaceData

/Content/DungeonsDash/Actors/Pickups/

BP_CardPickup_Base
BP_CardPickup_FloatingRune

/Content/DungeonsDash/Actors/Projectiles/

BP_Projectile_Base
BP_Projectile_Fireball
BP_Projectile_HomingArrow

/Content/DungeonsDash/Actors/Traps/

BP_Trap_Base
BP_Trap_ExplosiveBarrel

/Content/DungeonsDash/Actors/CardActions/

BP_Action_Base
BP_Action_SpeedPotion
BP_Action_Fireball
BP_Action_HomingArrow
BP_Action_ArcaneShield
BP_Action_ExplosiveBarrel
BP_Action_ComebackBlessing

/Content/DungeonsDash/Environment/Track/

BP_SurfaceZone_Base
BP_SurfaceZone_Mud
BP_SurfaceZone_Water
BP_BoostPad

/Content/DungeonsDash/UI/

WBP_CardHUD

5. Ajustes al sistema de cartas del equipo

La estructura del sistema de cartas propuesta por el equipo se mantiene: Data Asset para datos, UObject Blueprint para la accion, inventario que solicita uso, GameManager que valida y ejecuta en servidor, y Modifier Component que aplica cambios al vehiculo. Los siguientes ajustes solo generalizan esa logica para que funcione con todas las cartas, no solo con turbo.

Elemento actual	Cambio recomendado	Razon
DA_Card_Turbo	DA_Card_Base + DA_Card_SpeedPotion	Permite reutilizar el sistema para todas las cartas.
BP_Action_Turbo	BP_Action_Base + hijos especificos	Cada carta hereda de una accion base comun.
ExecuteAction	Mantener	La accion recibe todo lo necesario

Dungeons Dash - TDD Completo Blueprint-First

	ExecuteAction(InstigatorVehicle, GameManagerRef, CardData)	sin depender de casts especificos.
SpeedBoost directo	ModifierType = SpeedBoost	Permite usar un enum de modificadores reutilizable.
Variables Name/Icon/Duration/Intensity	Agregar CardDescription, CardType, Rarity, SpawnActorClass, TargetRule	Permite cartas de proyectil, trampa, escudo y recuperacion.

5.1 DA_Card_Base

Variable	Tipo	Uso
CardName	Text	Nombre visible en HUD.
CardDescription	Text	Descripcion corta de la carta.
CardIcon	Texture2D	Icono en el HUD.
CardType	E_CardType	Basic, Class o Track.
CardRarity	E_CardRarity	Common, Rare o Epic.
CardActionClass	Class Reference	Blueprint UObject que ejecuta la logica.
Duration	Float	Duracion del efecto.
Intensity	Float	Fuerza o intensidad del efecto.
Cooldown	Float	Opcional para balance.
SpawnActorClass	Class Reference	Proyectil o trampa si aplica.
ModifierType	E_DashModifierType	Tipo de modificador.
TargetRule	Enum opcional	Self, ClosestEnemy, FirstPlace, BehindPlayer.

6. Reglas corregidas del sistema de cartas

Las cartas se seleccionan solo con la cruceta izquierda/derecha. No existe descarte manual. Si el inventario esta lleno, el jugador debe usar una carta para liberar espacio.

Input	Accion
IA_SelectCardLeft	Seleccionar carta anterior
IA_SelectCardRight	Seleccionar carta siguiente
IA_UseCard	Usar carta seleccionada

No se debe crear IA_DiscardCard ni usar DiscardSelectedCard durante el MVP.

7. Blueprint principal del vehiculo

Blueprint	Clase padre	Ubicacion
BP_DashVehicle_Base	ChaosWheeledVehiclePawn	/Content/DungeonsDash/Actors/Vehicles/
BP_DashVehicle_Knight	BP_DashVehicle_Base	Variante de clase
BP_DashVehicle_Mage	BP_DashVehicle_Base	Variante de clase
BP_DashVehicle_Barbarian	BP_DashVehicle_Base	Variante de clase

Dungeons Dash - TDD Completo Blueprint-First

El vehiculo debe construirse sobre Chaos Vehicles porque el sistema requiere aceleracion, frenado, giro y comportamiento de ruedas. El Blueprint base contiene referencias principales y delega sistemas complejos a componentes.

8. Componentes de BP_DashVehicle_Base

Componente	Nombre recomendado	Uso
Skeletal Mesh	SK_VehicleBody	Cuerpo fisico y visual del vehiculo.
Chaos Vehicle Movement	VehicleMovement	Aceleracion, frenado, giro, ruedas y suspension.
Spring Arm	SpringArm_Camera	Camara arcade suavizada.
Camera	FollowCamera	Vista del jugador.
Arrow Component	ForwardArrow	Direccion frontal para boosts y proyectiles.
Scene Component	SC_ProjectileSpawnPoint	Punto de spawn de proyectiles.
Scene Component	SC_BackSpawnPoint	Punto de spawn de trampas.
Niagara Component	NS_BoostTrail	Estela de boost.
Niagara Component	NS_DriftSmoke_Left / Right	Humo de drift.
Niagara Component	NS_ShieldAura	Aura del escudo.
Audio Component opcional	AC_EngineLoop	Sonido del motor.

9. Componentes Blueprint del vehiculo

9.1 BPC_CardInventory

Componente encargado de guardar cartas, seleccionarlasy solicitar al servidor el uso de la carta actual. No sabe que hace cada carta.

Variable	Tipo	Uso
CardSlots	Array DA_Card_Base	Cartas actuales.
MaxCardSlots	Integer	Capacidad maxima, recomendado 3.
SelectedCardIndex	Integer	Slot seleccionado.
OwningVehicle	Actor Reference	Vehiculo propietario.

```
AddCard(CardData) -> Boolean  
SelectNextCard()  
SelectPreviousCard()  
UseCurrentCard()  
Server_RequestUseCard(CardIndex)  
OnRep_CardSlots()  
OnRep_SelectedCardIndex()
```

9.2 BPC_ModifierComponent

Componente que aplica modificadores al vehiculo y replica estados para activar efectos visuales en clientes.

Variable	Tipo	Replicacion
bIsTurboActive	Boolean	RepNotify
bHasShield	Boolean	RepNotify

Dungeons Dash - TDD Completo Blueprint-First

bIsInvincible	Boolean	RepNotify
bIsStunned	Boolean	RepNotify
CurrentSpeedMultiplier	Float	Replicated
CurrentTorqueMultiplier	Float	Replicated
CurrentHandlingMultiplier	Float	Replicated
ShieldHitsRemaining	Integer	Replicated

ApplyModifier(ModifierType, Duration, Intensity)

RemoveModifier(ModifierType)

ApplyForwardBoost(BoostForce)

ApplyExternalBoost(BoostForce, Direction)

ApplyHitImpulse(Direction, Force)

ApplyTemporaryControlLoss(Duration)

ActivateShield(Duration, Hits)

SetInvincible(Duration)

9.3 BPC_VehicleDrift

Componente encargado del drift, acumulacion de carga y mini-turbo.

Variable	Tipo	Valor inicial sugerido
bIsDrifting	Boolean	false
DriftCharge	Float	0
MaxDriftCharge	Float	100
DriftChargeRate	Float	35
MiniTurboThreshold	Float	60
MiniTurboForce	Float	70000
MinSpeedForDrift	Float	800
MinSteerForDrift	Float	0.35

StartDrift()

UpdateDrift(DeltaSeconds)

EndDrift()

ActivateMiniTurbo()

9.4 BPC_VehicleSurfaceHandler

Componente encargado de aplicar efectos de lodo, agua y superficies especiales.

Variable	Tipo	Uso
CurrentSurfaceType	E_DashSurfaceType	Superficie actual.
NormalHandlingMultiplier	Float	Control normal.
MudHandlingMultiplier	Float	Control en lodo.
WaterHandlingMultiplier	Float	Control en agua.
CurrentSurfaceSpeedMultiplier	Float	Multiplicador de velocidad actual.

ApplySurfaceEffect(SurfaceType, FrictionMultiplier, SpeedMultiplier)

ResetSurfaceEffect()

9.5 BPC_VehicleRaceData

Componente opcional para guardar vuelta, checkpoint, posicion y respawn.

Dungeons Dash - TDD Completo Blueprint-First

Variable	Tipo	Uso
CurrentLap	Integer	Vuelta actual.
CurrentCheckpointIndex	Integer	Ultimo checkpoint valido.
DistanceToNextCheckpoint	Float	Ordenamiento de carrera.
RacePosition	Integer	Posicion actual.
LastCheckpointTransform	Transform	Respawn.

10. Enhanced Input

Input Action	Tipo	Uso
IA_Accelerate	Axis 1D	Acelerar
IA_Brake	Axis 1D	Frenar
IA_Steer	Axis 1D	Girar
IA_Jump	Boolean	Saltar
IA_Drift	Boolean	Derrapar
IA_UseCard	Boolean	Usar carta
IA_SelectCardLeft	Boolean	Carta anterior
IA_SelectCardRight	Boolean	Carta siguiente

11. Movimiento base del vehiculo

11.1 Acelerar

IA_Accelerate Triggered

-> Si bIsStunned == false

-> FinalThrottle = ThrottleValue * CurrentSpeedMultiplier * CurrentSurfaceSpeedMultiplier

-> VehicleMovement Set Throttle Input(FinalThrottle)

11.2 Frenar

IA_Brake Triggered

-> Si bIsStunned == false

-> VehicleMovement Set Brake Input(BrakeValue)

11.3 Girar

IA_Steer Triggered

-> Si bIsStunned == false

-> FinalSteering = SteerValue * CurrentHandlingMultiplier

-> VehicleMovement Set Steering Input(FinalSteering)

12. Sistema de salto

El salto permite evadir obstaculos, esquivar ataques y alcanzar rutas elevadas. Debe sentirse fisico y arcade.

Variable	Tipo	Valor inicial sugerido
JumpForce	Float	45000 - 70000
bCanJump	Boolean	true
bIsGrounded	Boolean	true/false

Dungeons Dash - TDD Completo Blueprint-First

JumpCooldown	Float	1.0
AirControlMultiplier	Float	0.4

CheckGrounded()

- > Line Trace Down desde el vehiculo
- > Si golpea Track/WorldStatic: bIsGrounded = true
- > Si no: bIsGrounded = false

TryJump()

- > Branch bCanJump
- > Branch bIsGrounded
- > Add Impulse Z = JumpForce
- > Set bCanJump = false
- > Set Timer ResetJump

ResetJump()

- > Set bCanJump = true

13. Drift y mini-turbo

El drift inicia cuando el jugador mantiene el input de derrape, esta girando, tiene suficiente velocidad y esta tocando el suelo. Mientras derrapa, acumula carga. Al soltar el drift, si la carga supera el umbral, se activa mini-turbo.

Condicion para iniciar drift:

```
IA_Drift Held == true
AND Abs(SteeringInput) >= MinSteerForDrift
AND CurrentSpeed >= MinSpeedForDrift
AND bIsGrounded == true
```

StartDrift()

- > Set bIsDrifting = true
- > Set DriftCharge = 0
- > Activate Drift Smoke

UpdateDrift(DeltaSeconds)

- > Si bIsDrifting
- > DriftCharge += DriftChargeRate * DeltaSeconds
- > Clamp DriftCharge 0-MaxDriftCharge
- > Si DriftCharge >= MiniTurboThreshold: bMiniTurboReady = true

EndDrift()

- > Set bIsDrifting = false
- > Deactivate Drift Smoke
- > Si bMiniTurboReady: ActivateMiniTurbo
- > Reset DriftCharge

ActivateMiniTurbo()

- > Get Actor Forward Vector
- > Add Impulse ForwardVector * MiniTurboForce
- > Set bIsTurboActive = true

Dungeons Dash - TDD Completo Blueprint-First

- > RepNotify activa VFX/SFX en clientes
- > Timer para apagar turbo

14. Terreno especial

Enum E_DashSurfaceType	Uso
Normal	Terreno normal.
Mud	Lodo, reduce velocidad y control.
Water	Agua, reduce velocidad y agarre.
BoostPad	Impulso instantaneo.
Ice	Opcional futuro.

14.1 BP_SurfaceZone_Base

Componente / Variable	Tipo	Uso
Box Collision	Componente	Detectar vehiculo.
SurfaceType	E_DashSurfaceType	Tipo de superficie.
FrictionMultiplier	Float	Multiplicador de friccion.
SpeedMultiplier	Float	Multiplicador de velocidad.
HandlingMultiplier	Float	Multiplicador de control.

OnComponentBeginOverlap

- > Check Other Actor has BPC_VehicleSurfaceHandler
- > ApplySurfaceEffect(SurfaceType, FrictionMultiplier, SpeedMultiplier)

OnComponentEndOverlap

- > ResetSurfaceEffect()

14.2 Lodo y agua

Zona	SpeedMultiplier	HandlingMultiplier
BP_SurfaceZone_Mud	0.75	0.65
BP_SurfaceZone_Water	0.70	0.60

En Blueprint puro se recomienda simular la perdida de agarre multiplicando el input de steering y throttle, en lugar de modificar cada rueda en runtime durante el MVP.

14.3 Boost Pad

Componente / Variable	Uso
Static Mesh	Visual del pad.
Box Collision	Overlap con vehiculo.
Arrow Component	Direccion del boost.
Niagara Component	Efecto visual.
BoostForce	Fuerza del impulso.
BoostDirectionMode	Usar direccion del pad o del vehiculo.

OnComponentBeginOverlap

- > Get direccion del Arrow
- > Call ApplyExternalBoost(BoostForce, Direction)

- > Play Sound
- > Spawn VFX

15. Actores de recoleccion de cartas

15.1 BP_CardPickup_Base

Componente	Uso
Scene Root	Raiz del actor.
Static Mesh	Visual: runa, cofre, cristal o pergamino.
Sphere Collision	Detecta al jugador.
Rotating Movement	Rotacion flotante sin usar Tick.
Niagara Component	Aura magica.
Audio opcional	Sonido de pickup y respawn.

Variable	Tipo	Uso
bIsActive	Boolean	Indica si puede entregar carta.
bCanRespawn	Boolean	Define si reaparece.
RespawnTime	Float	Tiempo de reaparicion.
bGiveRandomCard	Boolean	Carta aleatoria o fija.
FixedCard	DA_Card_Base	Carta fija.
PossibleCards	Array DA_Card_Base	Lista de cartas posibles.
PickupVFX	Niagara System	Efecto al recoger.
PickupSound	Sound Base	Sonido al recoger.

OnComponentBeginOverlap

- > Branch bIsActive
- > GetCardToGive
- > Other Actor Get BPC_CardInventory o BPI_CardInventoryReceiver
- > AddCard(CardData)
- > Si AddCard == true: DeactivatePickup

DeactivatePickup()

- > Set bIsActive = false
- > Hide Mesh
- > Collision = NoCollision
- > Deactivate Niagara
- > Spawn VFX / Play Sound
- > Si bCanRespawn: Set Timer RespawnPickup

RespawnPickup()

- > Set bIsActive = true
- > Show Mesh
- > Collision = QueryOnly
- > Activate Niagara

16. Inventario de cartas

El inventario guarda un maximo recomendado de 3 cartas. No permite descarte manual. La carta se usa mediante una solicitud al servidor para evitar inconsistencias o trampas.

AddCard(CardData) -> Boolean

- > Si CardSlots.Length < MaxCardSlots
- > Add CardData
- > OnCardAdded Dispatcher
- > Return true
- > Si no
- > Return false

SelectNextCard()

- > SelectedCardIndex = (SelectedCardIndex + 1) % CardSlots.Length

SelectPreviousCard()

- > SelectedCardIndex = (SelectedCardIndex - 1 + CardSlots.Length) % CardSlots.Length

UseCurrentCard()

- > Server_RequestUseCard(SelectedCardIndex)

Server_RequestUseCard(CardIndex)

- > Validar que CardIndex exista
- > Obtener CardData
- > Remover carta del inventario
- > Get GameMode / Cast GM_RacingGame
- > ProcessCardActivation(CardData, OwningVehicle)
- > Replicar CardSlots

17. GameManager / GameMode

El GameManager existe en servidor y es la autoridad que procesa activaciones de cartas. Esto mantiene el vehiculo simple y evita que cada carta tenga que resolver por su cuenta la carrera completa.

GM_RacingGame::ProcessCardActivation(CardData, InstigatorVehicle)

- > Validar CardData
- > Leer CardActionClass
- > Construct Object from Class(CardActionClass)
- > ExecuteAction(InstigatorVehicle, Self, CardData)

Funcion auxiliar	Uso
GetFirstPlaceVehicle()	Cartas que atacan al lider.
GetClosestEnemyAhead(Instigator, Radius)	Proyectiles guiados.
GetVehicleBehind(Instigator)	Robo o cartas defensivas futuras.
GetAllRacingVehicles()	Efectos caoticos o de area.
GetPlayerRacePosition(Vehicle)	Bendicion de recuperacion.

18. Acciones de carta

18.1 BP_Action_Base

Todas las cartas heredan de BP_Action_Base. Debe ser un UObject Blueprint porque la accion no necesita existir fisicamente en el mundo, salvo que spawnee un actor como proyectil o trampa.

BP_Action_Base

ExecuteAction(InstigatorVehicle, GameManagerRef, CardData)

- Usar UObject para turbo, escudo, invencibilidad, recuperacion y cartas de reglas.
- Usar Actor para proyectiles, trampas, explosiones o elementos fisicos en pista.
- La accion puede pedir informacion al GameManager y aplicar modificadores al vehiculo.

19. Sistema de modificadores

E_DashModifierType	Uso
SpeedBoost	Aumenta velocidad o torque.
HandlingBoost	Mejora control.
Stun	Bloquea control temporalmente.
Shield	Bloquea impactos.
Invincibility	Ignora efectos negativos.
Slow	Reduce velocidad.
Knockback	Empuje por impacto.
SurfaceSlow	Penalizacion de terreno.

ApplyModifier(ModifierType, Duration, Intensity)

-> Switch ModifierType

SpeedBoost:

- > CurrentSpeedMultiplier = Intensity
- > bIsTurboActive = true
- > Timer Remove SpeedBoost

Shield:

- > bHasShield = true
- > ShieldHitsRemaining = Intensity
- > Timer Remove Shield

Stun:

- > bIsStunned = true
- > Timer Remove Stun

OnRep_bIsTurboActive

- > True: Activate NS_BoostTrail + Play Sound
- > False: Deactivate NS_BoostTrail

OnRep_bHasShield

- > True: Activate NS_ShieldAura
- > False: Deactivate NS_ShieldAura

20. Power-ups basicos del MVP

Carta	Rol	Implementacion
Pocion de velocidad	Boost	UObject Action + Modifier
Bola de fuego	Proyectil recto	UObject Action + Projectile Actor
Flecha encantada	Proyectil guiado	UObject Action + Homing Projectile
Escudo arcano	Defensa	UObject Action + Modifier
Barril explosivo	Trampa	UObject Action + Trap Actor
Bendicion de recuperacion	Comeback	UObject Action + Modifier

21. Carta 1 - Pocion de velocidad

Elemento	Valor
Data Asset	DA_Card_SpeedPotion
Action	BP_Action_SpeedPotion
CardName	Pocion de velocidad
CardType	Basic
Duration	3.0
Intensity	1.5
ModifierType	SpeedBoost
Resultado	Aumenta velocidad/torque durante unos segundos.

ExecuteAction

- > InstigatorVehicle Get BPC_ModifierComponent
- > ApplyModifier(SpeedBoost, CardData.Duration, CardData.Intensity)

22. Carta 2 - Bola de fuego

Elemento	Valor
Data Asset	DA_Card_Fireball
Action	BP_Action_Fireball
CardName	Bola de fuego
CardType	Basic
SpawnActorClass	BP_Projectile_Fireball
Intensity	HitForce
Resultado	Proyectil recto hacia adelante.

ExecuteAction

- > Get ProjectileSpawnPoint del vehiculo
- > SpawnActor BP_Projectile_Fireball
- > Set Owner = InstigatorVehicle

BP_Projectile_Fireball OnOverlap

- > Si OtherActor != Owner
- > ApplyHitImpulse(Direction, Force)
- > Spawn Impact VFX
- > Destroy Actor

23. Carta 3 - Flecha encantada

Elemento	Valor
Data Asset	DA_Card_HomingArrow
Action	BP_Action_HomingArrow
CardName	Flecha encantada
CardType	Basic
SpawnActorClass	BP_Projectile_HomingArrow
SearchRadius	3000
Resultado	Proyectil guiado hacia rival delante.

ExecuteAction

- > GameManagerRef GetClosestEnemyAhead(InstigatorVehicle, SearchRadius)
- > Spawn BP_Projectile_HomingArrow
- > Set TargetVehicle

BP_Projectile_HomingArrow

- > Projectile Movement Is Homing = true
- > Homing Target Component = TargetVehicle Mesh

24. Carta 4 - Escudo arcano

Elemento	Valor
Data Asset	DA_Card_ArcaneShield
Action	BP_Action_ArcaneShield
CardName	Escudo arcano
CardType	Basic
Duration	5.0
Intensity	1 golpe
ModifierType	Shield
Resultado	Bloquea un impacto.

ExecuteAction

- > InstigatorVehicle Get BPC_ModifierComponent
- > ApplyModifier(Shield, Duration, Intensity)

ApplyHitImpulse

- > Si bHasShield: consumir escudo e ignorar golpe
- > Si no: aplicar golpe normal

25. Carta 5 - Barril explosivo

Elemento	Valor
Data Asset	DA_Card_ExplosiveBarrel
Action	BP_Action_ExplosiveBarrel
CardName	Barril explosivo
CardType	Basic
SpawnActorClass	BP_Trap_ExplosiveBarrel
Intensity	ExplosionForce
Duration	LifeTime
Resultado	Trampa colocada atras del vehiculo.

Dungeons Dash - TDD Completo Blueprint-First

ExecuteAction

- > Get BackSpawnPoint del vehiculo
- > SpawnActor BP_Trap_ExplosiveBarrel
- > Set Owner = InstigatorVehicle

BP_Trap_ExplosiveBarrel

- > BeginPlay: bArmed = false, Timer ArmTrap
- > OnOverlap: si bArmed y OtherActor != Owner, Explode
- > Explode: aplicar impulso a vehiculos cercanos y destruir actor

26. Carta 6 - Bendicion de recuperacion

Elemento	Valor
Data Asset	DA_Card_ComebackBlessing
Action	BP_Action_ComebackBlessing
CardName	Bendicion de recuperacion
CardType	Basic
Duration	2.5
Intensity	BaseBoost
Resultado	Boost mayor si va rezagado.

ExecuteAction

- > GameManagerRef GetPlayerRacePosition(InstigatorVehicle)
- > Calcular multiplicador segun posicion
- > ApplyModifier(SpeedBoost, Duration, FinalIntensity)

Multiplicadores sugeridos:

- 1er lugar: 1.0
- Posicion media: 1.3
- Ultimo lugar: 1.7

27. Interfaces Blueprint

Interface	Funcion	Uso
BPI_VehicleEffects	ApplyForwardBoost(BoostForce)	Power-ups de velocidad.
BPI_VehicleEffects	ApplyExternalBoost(BoostForce, Direction)	Boost Pads.
BPI_VehicleEffects	ApplyHitImpulse(Direction, Force)	Proyectiles y explosiones.
BPI_VehicleEffects	ApplyTemporaryControlLoss(Duration)	Stun o perdida de control.
BPI_VehicleEffects	GetProjectileSpawnPoint()	Spawn de proyectiles.
BPI_VehicleEffects	GetBackSpawnPoint()	Spawn de trampas.
BPI_CardInventoryReceiver	ReceiveCard(CardData) -> Boolean	Pickups entregan cartas.
BPI_HitReceiver	ReceivePowerUpHit(HitData, Instigator)	Impactos de proyectiles/trampas.

28. Flujo completo: recoger carta

Jugador pasa por BP_CardPickup_FloatingRune

- > Sphere Collision detecta overlap
- > Pickup verifica bIsActive
- > Pickup selecciona carta
- > Llama ReceiveCard en vehiculo/inventario
- > BPC_CardInventory intenta AddCard
- > Si hay espacio:
 - > Añade carta
 - > Actualiza HUD
 - > Pickup se desactiva
 - > Timer de respawn
- > Si no hay espacio:
 - > No pasa nada

29. Flujo completo: usar carta

Jugador presiona IA_UseCard

- > BPC_CardInventory UseCurrentCard
- > Server_RequestUseCard
- > Servidor valida carta
- > Remueve carta del inventario
- > GM_RacingGame ProcessCardActivation
- > Construct Object from Class CardActionClass
- > ExecuteAction
- > La accion aplica modificador, spawna proyectil o coloca trampa
- > RepNotify actualiza VFX/SFX en clientes

30. Flujo completo: recibir ataque

Proyectil / trampa golpea vehiculo

- > Verifica que no sea el Owner
- > Busca BPC_ModifierComponent
- > Si bIsInvincible: ignorar
- > Si bHasShield: consumir escudo y destruir proyectil
- > Si no: ApplyHitImpulse y opcional ApplyTemporaryControlLoss
- > RepNotify activa VFX de golpe

31. Multiplayer / red LAN

Regla principal: el servidor es la autoridad. El cliente puede presionar input, mostrar HUD y solicitar usar cartas. El servidor valida cartas, aplica modificadores, spawna proyectiles/trampas, calcula objetivos y replica estados importantes.

Elemento	Tipo	Uso
Server_RequestUseCard(CardIndex)	RPC Run on Server	Uso de cartas.
Server_RequestJump()	RPC opcional	Salto autoritativo si se requiere.
Server_RequestDriftStart/End	RPC opcional	Drift autoritativo si se requiere.
bIsTurboActive	RepNotify	VFX/SFX de boost.
bHasShield	RepNotify	Aura de escudo.

Dungeons Dash - TDD Completo Blueprint-First

bIsInvincible	RepNotify	Efecto de invencibilidad.
bIsStunned	RepNotify	Feedback de aturdimiento.

32. HUD de cartas

El HUD debe mostrar tres slots, icono de cada carta, slot seleccionado y feedback al usar carta. Debe actualizarse cuando se agrega una carta, se usa una carta o cambia la seleccion.

WBP_CardHUD

- > Slot 0 Icon
- > Slot 1 Icon
- > Slot 2 Icon
- > Selected Slot Highlight

Event Dispatchers en BPC_CardInventory:

- OnCardAdded
- OnCardUsed
- OnSelectedCardChanged
- OnInventoryChanged

33. Estados del vehiculo y respawn

E_DashVehicleState	Uso
Normal	Control normal.
Drifting	Esta derrapando.
Boosting	Tiene impulso activo.
Airborne	Esta en el aire.
Stunned	Ignora input temporalmente.
Respawning	Reposicionando en checkpoint.
Finished	Termino la carrera.

33.1 RespawnAtLastCheckpoint

RespawnAtLastCheckpoint()

- > Set State = Respawning
- > Disable input
- > ResetVehiclePhysics
- > Set Actor Transform = LastCheckpointTransform
- > Enable input
- > Set State = Normal

ResetVehiclePhysics()

- > Set Physics Linear Velocity = 0
- > Set Physics Angular Velocity = 0

34. Orden recomendado de implementacion

1. Crear BP_DashVehicle_Base y configurar Chaos Vehicle Movement.
2. Crear IMC_VehicleControls e Input Actions.

3. Implementar acelerar, frenar y girar.
4. Agregar camara arcade.
5. Crear BPC_ModifierComponent.
6. Implementar salto.
7. Crear BPC_VehicleDrift con drift y mini-turbo.
8. Crear DA_Card_Base y BP_Action_Base.
9. Crear BPC_CardInventory con seleccion izquierda/derecha y uso obligatorio.
10. Crear GM_RacingGame::ProcessCardActivation.
11. Crear BP_CardPickup_Base y BP_CardPickup_FloatingRune.
12. Crear cartas: SpeedPotion, Fireball, ArcaneShield, ExplosiveBarrel, HomingArrow y ComebackBlessing.
13. Crear BP_SurfaceZone_Mud, BP_SurfaceZone_Water y BP_BoostPad.
14. Conectar HUD con Event Dispatchers o RepNotify.
15. Probar carrera completa y balancear valores.

35. Responsabilidades finales por sistema

Sistema	Responsabilidad
BP_DashVehicle_Base	Movimiento y referencias principales.
BPC_CardInventory	Guardar, seleccionar y solicitar uso de cartas.
DA_Card_Base	Datos pasivos de carta.
BP_Action_Base	Logica ejecutable de carta.
GM_RacingGame	Validacion y resolucion en servidor.
BPC_ModifierComponent	Estados, boosts, golpes y RepNotify.
BPC_VehicleDrift	Drift, carga y mini-turbo.
BPC_VehicleSurfaceHandler	Lodo, agua y superficies.
BP_CardPickup_Base	Entregar cartas al jugador.
BP_Projectile_*	Ataques fisicos.
BP_Trap_*	Trampas de pista.
WBP_CardHUD	Mostrar cartas y seleccion.

36. Conclusion tecnica

Este TDD unifica vehiculos, cartas, pickups, power-ups, terreno y red bajo una arquitectura Blueprint-first. La logica del sistema de cartas se mantiene, pero se generaliza para permitir cartas de turbo, proyectil, escudo, trampa, ataque guiado y recuperacion. El vehiculo no depende directamente de las cartas: expone funciones y componentes que pueden ser usados por el GameManager, los power-ups y las zonas de terreno.

La regla de gameplay queda integrada: las cartas se seleccionan solo con cruceta izquierda/derecha y no se descartan manualmente. El jugador debe usar una carta para liberar espacio. Con esta division, cada miembro del equipo puede implementar su parte sin bloquear a los demas.